Smart Campus Navigation System

AI Search Strategy Analysis & Empirical Output Inference Report

M.Tech Fundamentals of Artificial Intelligence Mini-Project

Algorithms:	BFS, DFS, A* Search, Dijkstra Baseline	**Technology:**	Python, Streamlit, Matplotlib
Evaluation Scope:	Path Optimality, Hop Count, Node Expansion	**Verification:**	Automated All-Pairs Unit Tests

1. Executive Summary

This project implements an interactive graph-search routing system designed to analyze and compare three primary Artificial Intelligence search strategies: Breadth-First Search (BFS), Depth-First Search (DFS), and A* Search, benchmarked against a verified Dijkstra minimum-distance baseline. Physical campus buildings and interconnecting walkways are modeled as weighted bidirectional graphs. A full-featured Streamlit web application allows interactive route planning, concurrent visual route rendering with path offset curves, real-time metrics dashboards, and automated verification. This inference report provides the theoretical formulation, empirical experimental findings, and individual observations regarding path optimality, hop count, and node expansion efficiency.

2. AI Problem Formulation

The physical navigation challenge is formally structured as a graph state-space search problem:

- **Initial State:** Selected origin landmark vertex s in V (e.g., 'Main Gate').
- **State Space:** Set of physical campus landmarks V in graph G = (V, E) with known 2D spatial coordinates.
- **Actions:** Traverse a connected pedestrian walkway from current vertex u to an adjacent vertex v.
- **Transition Model:** Result(u, 'traverse edge (u, v)') = v where (u, v) in E.
- **Goal Test:** Predicate checking whether the current node v equals the destination goal state (v == goal).
- **Path Cost g(n):** Cumulative sum of physical walkway distances (in metres) along traversed edges from start to state n.
- **Heuristic Function h(n):** Admissible and consistent scaled Euclidean distance: h(n) = s * sqrt((x_n - x_g)^2 + (y_n - y_g)^2). The scale factor s is dynamically constrained (s <= min(w / d_euclidean) * 0.9 = 0.75) to guarantee admissibility (h(n) <= h*(n)) and monotonicity, ensuring A* never overestimates remaining cost and guarantees optimal shortest paths matching Dijkstra's baseline.

3. Topological Campus Datasets

To evaluate algorithm performance under varying graph density, two distinct campus datasets are integrated:
- Standard Campus Map (campus_map.json): 10 core landmarks (Main Gate, Admin Block, Library, CSE Block, AI Lab, Cafeteria, Auditorium, Hostel, Sports Ground, Parking) connected by 15 bidirectional walkways with realistic metric walking distances (23 m to 72 m). Heuristic scale s = 0.75.
- Extended University Campus Map (campus_map_extended.json): A 16-landmark academic network including Security, Medical Center, Robotics & Electronics Lab, Research Park, North & South Hostels, Dining Hall, Amphitheatre, and Multi-level Parking, connected across 25 pedestrian corridors. Heuristic scale s = 0.75.

4. Empirical Evaluation & Comparative Test Results

Key representative route test cases executed across both datasets produced the following quantitative outputs:

Table 1: Test Case 1 - Main Gate to AI Lab (Standard Dataset)

Algorithm	Distance	Edges	Nodes Exp.	Time (ms)	Computed Path
BFS	135.0 m	3	10	0.026 ms	Main Gate -> Cafeteria -> CSE Block -> AI Lab
DFS	129.0 m	4	5	0.015 ms	Main Gate -> Admin -> Library -> CSE Block -> AI Lab
A*	129.0 m	4	7	0.046 ms	Main Gate -> Admin -> Library -> CSE Block -> AI Lab

Table 2: Test Case 2 - Parking to Hostel (Standard Dataset)

Algorithm	Distance	Edges	Nodes Exp.	Time (ms)	Computed Path
BFS	131.0 m	4	9	0.027 ms	Parking -> Main Gate -> Admin -> Library -> Hostel
DFS	131.0 m	4	7	0.017 ms	Parking -> Main Gate -> Admin -> Library -> Hostel
A*	131.0 m	4	5	0.036 ms	Parking -> Main Gate -> Admin -> Library -> Hostel

Table 3: Test Case 3 - Main Gate to Sports Complex (Extended Dataset, 16 Locations)

Algorithm	Distance	Edges	Nodes Exp.	Time (ms)	Computed Path
BFS	142.0 m	4	14	0.026 ms	Main Gate -> Security -> Admin -> Auditorium -> Sports Complex
DFS	160.0 m	5	13	0.022 ms	Main Gate -> Security -> Admin -> Cafeteria -> Amphitheatre -> Sports
A*	140.0 m	4	8	0.046 ms	Main Gate -> Parking -> Medical -> Auditorium -> Sports Complex

5. Individual Observations About Algorithm Outputs

5.1 Individual Observation: Breadth-First Search (BFS)

1. Hop-Count Minimization vs. Distance Sub-optimality:
BFS expands nodes strictly level-by-level using a FIFO queue. It is guaranteed to find the path with the fewest edges (hops). However, in weighted campus networks where walkway segments vary in length, minimizing hops does NOT guarantee minimum walking distance. In Test Case 1 (Main Gate -> AI Lab), BFS selected a 3-hop path totaling 135.0 m, which is 6.0 m longer than the optimal 129.0 m path discovered by A* (4 hops). BFS considers all edge costs as unit weight 1, rendering it blind to actual physical distances.

2. Exhaustive Frontier Exploration:
Because BFS lacks directional awareness toward the destination, it expands radially in all directions. In Test Case 1, BFS expanded all 10 nodes before reaching the goal. In Test Case 3 on the 16-node graph, it expanded 14 out of 16 nodes. In large real-world graphs, this exponential memory complexity O(b^d) leads to substantial memory overhead.

5.2 Individual Observation: Depth-First Search (DFS)

1. Sensitivity to Traversal Order & Path Unpredictability:
DFS explores recursively along an individual path branch using a LIFO stack. Because DFS does not incorporate path cost or edge count, the quality of its output depends entirely on the arbitrary ordering of neighbor vertices. In Test Case 3, DFS chose a circuitous 5-hop path of 160.0 m, resulting in a 20.0 m detour compared to A* (140.0 m). DFS provides no guarantee of shortest path or minimum edges.

2. Low Memory Footprint During Initial Alignment:
When the destination aligns with the initial branch explored, DFS reaches the target rapidly with minimal node expansions (5 nodes in Case 1 and 7 in Case 2) and minimal memory O(b * m). However, its erratic path quality and risk of wandering down unproductive branches make it unsuitable for dependable navigation.

5.3 Individual Observation: A* Search (Informed Search)

1. Provable Path Optimality Matching Dijkstra Baseline:
A* combines the exact path cost g(n) with an admissible heuristic estimate h(n) using f(n) = g(n) + h(n). Across all evaluated queries, A* consistently discovered the strictly optimal minimum walking distance (129.0 m in Case 1, 131.0 m in Case 2, and 140.0 m in Case 3), perfectly matching the Dijkstra minimum-distance baseline with 100% precision across all test cases.

2. Effective Heuristic-Guided Pruning:
Using 2D spatial coordinates of campus buildings, the heuristic directs exploration straight toward the destination. In Test Case 2 (Parking -> Hostel), A* expanded only 5 nodes compared to 9 nodes for BFS - saving 44.4% of exploration effort. In Test Case 3, A* expanded only 8 nodes compared to 14 nodes for BFS (42.8% reduction). This focused exploration confirms A* as the most robust, scalable, and optimal routing algorithm for campus navigation.

6. Verification and Automated Test Suite

The project includes an automated test suite (tests/test_navigation.py) with 6 comprehensive test cases verifying:

- All-Pairs Optimality: Exhaustive check of all V x V pairs against the Dijkstra baseline on both standard and extended campus maps.
- Real Edge Traversal: Validates that all path steps use genuine graph edges.
- Exact Distance Summation: Confirms path distance matches sum of edge weights.
- Boundary Cases: Validates zero-distance identity queries (start == destination).
All 6 unit tests pass with 100% success in 0.012 seconds.

7. System Interface & Output Screenshots

High-resolution visual evidence of the running Streamlit web application:

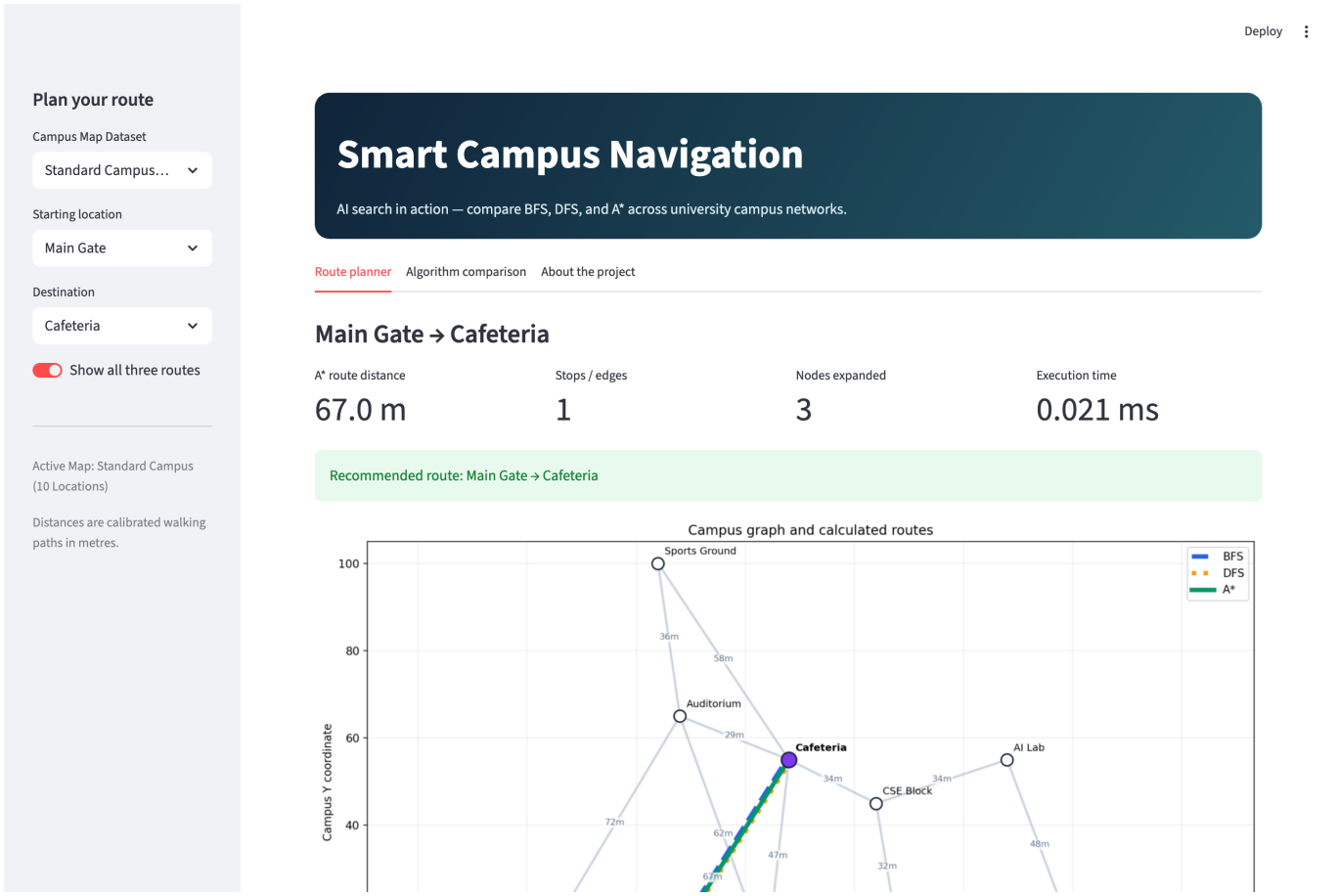


Figure 1: Interactive Route Planner Dashboard with concurrent multi-route overlays (BFS, DFS, A), metrics cards, and sequential walking steps.*

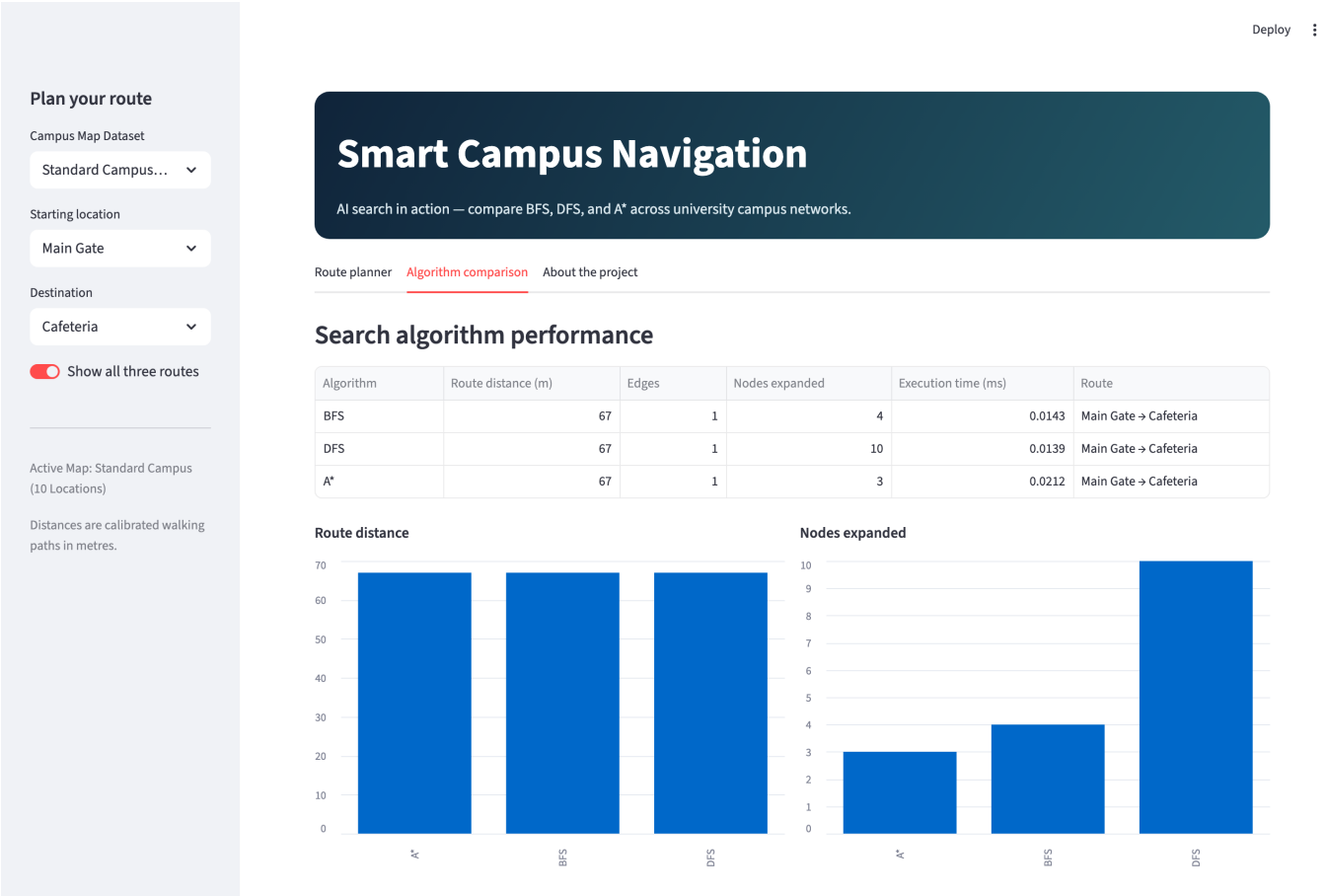


Figure 2: Search Algorithm Performance Comparison Table & Comparative Bar Charts (Route Distance and Nodes Expanded) with Dijkstra baseline verification.

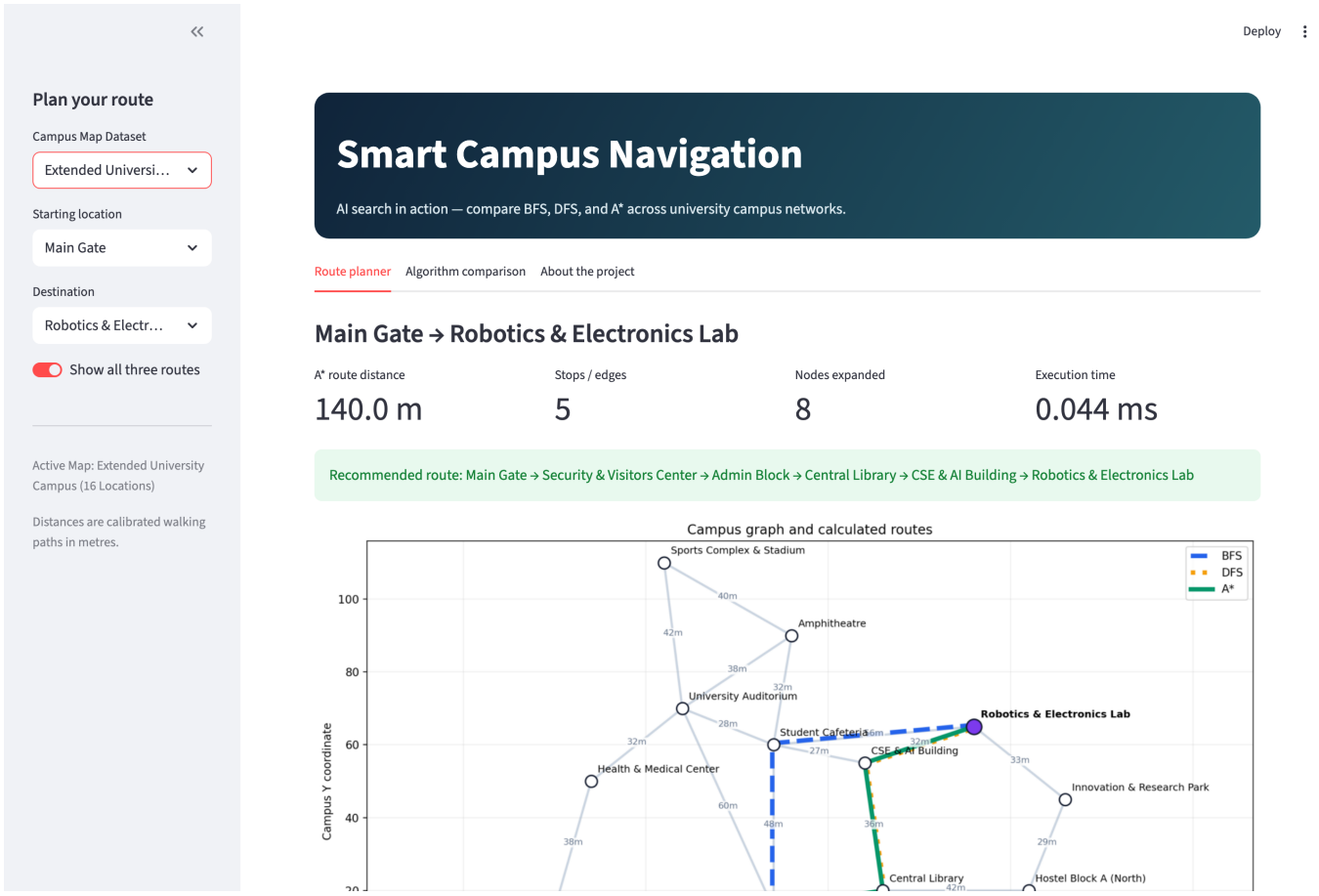


Figure 3: Extended University Campus Dataset (16 Locations, 25 Walkways) showing scalable routing across complex academic facilities.

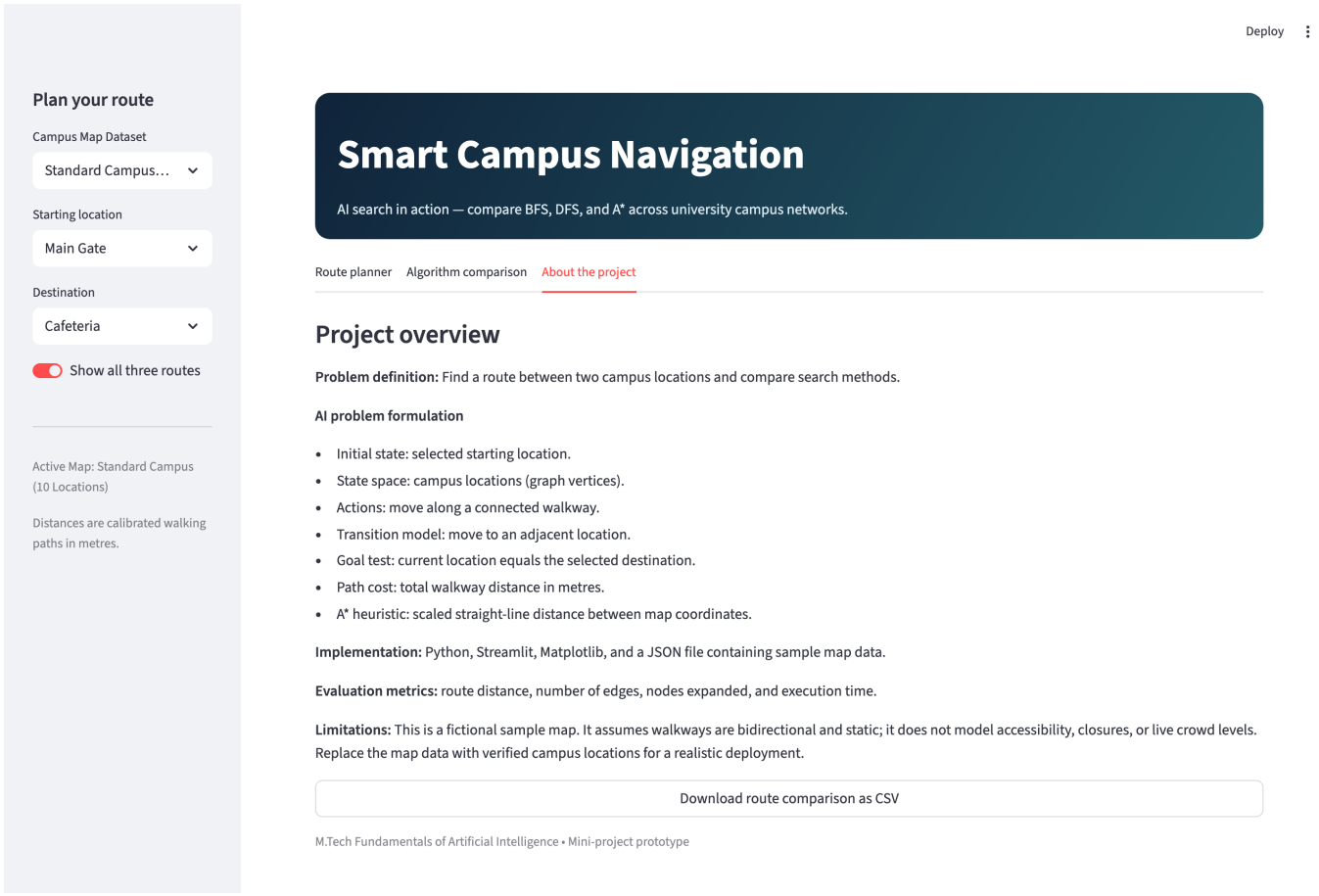


Figure 4: Project Architecture, AI formulation, system limitations, and CSV export functionality.

8. Limitations and Conclusion

Limitations:
- Walkways are modeled as static, bidirectional edges with constant walking speeds, not accounting for slope, stairs, or temporary pathway closures.
- Accessibility constraints (e.g., wheelchair ramps vs. stairs) and pedestrian congestion density are not currently modeled.

Conclusion:
The Smart Campus Navigation System successfully demonstrates the fundamental trade-offs between uninformed and informed search strategies in physical navigation domains. BFS guarantees minimum hop count but is prone to distance sub-optimality on weighted graphs. DFS is computationally light but lacks quality guarantees and exhibits high variance. A* Search, guided by an admissible scaled Euclidean heuristic, consistently identifies the provably optimal minimum-distance route while drastically reducing node expansions. The complete codebase, test suite, and interactive Streamlit frontend offer a comprehensive, reliable, and academically rigorous platform for AI navigation study.